



INNOPOLIS
UNIVERSITY

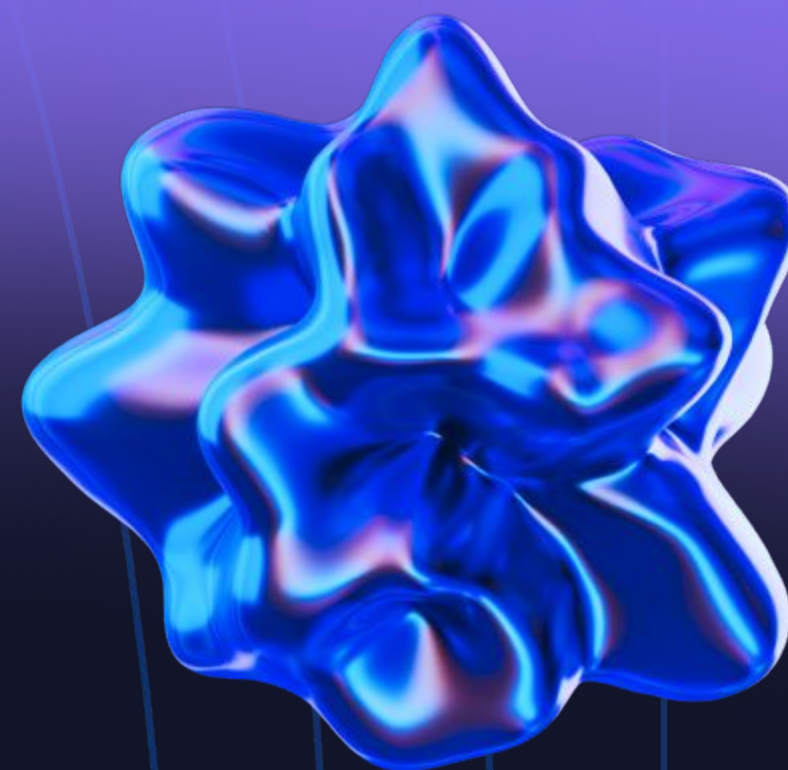
Student: Mikhail Voronin

Supervisor: Artem Burmyakov

Consultant: Alexander Tormasov

June 2026

Research and Adaptation of Components from the Persistent OS Phantom for Genode



Relevance & Objective

Why it matters

- Conception developed in the 1970s
- The hardware and network balances have changed
- Persistent data storage must be protected
- Snapshot security in Phantom was not considered

Why now?

It is necessary to examine the relevance of persistence concepts in the current environment

Goals

- Analyze computer usage scenarios
- Develop security assurance component
- Experimentally check protection correctness

Current projects

Different projects are implementing the concept PhantomOS is our choice. The issue we solve is that snapshots are stored in plaintext

Persistence is the period of time during which an object is usable¹

Two-level store (today's dominant model, non-persistent)

- Data is either active (RAM) or stored (disk)
- Programmer manually moves data between levels

Drawbacks

- Data loss
- I/O boilerplate ($\approx 30\%$ of code)
- Frequent initialization and configuration

Alternative: the environment manages persistence

Orthogonal Persistence Principles²

1. Independence
2. Data type orthogonality
3. Identification

Result

No two-level store drawbacks, and improved disk bandwidth utilization

¹M. P. Atkinson, P. J. Bailey, K. J. Chisholm, W. P. Cockshott, R. Morrison. "PS-algol: A language for persistent programming", 1983

²R. Morrison, M. P. Atkinson. "Persistent Languages and Architectures", 1990

PhantomOS¹ is a non-Unix-like OS implementing orthogonal persistence²

PhantomOS ported to the **Genode OS C++ framework³**
This is our test platform

PhantomOS features

- Phantom Virtual Machine (PVM): bytecode VM of a persistent language
- Simplified POSIX layer (non-persistent)
- Periodical snapshots via **Copy-on-Write** (implementation of persistence)
- Changed pages written to disk (incremental)
- Programs temporarily suspended during snapshot preparation

Why Genode

- Proven microkernels: NOVA, seL4, Fiasco.OC
- Capability-based security
- Minimal trusted computing base (TCB)
- Support modern hardware

¹<http://phantomos.org/>

²D. Zavalishin. PhantomOS, 2010

³<https://genode.org/>

Snapshot mechanism & Problem Statement

Snapper¹ is a file-system-based transactional snapshot mechanism²

Previous approach: “superblock”

- Snapshots stored as monolithic objects
- Fast, but no fault tolerance and no storage recovery

Problem

Snapshots contain **sensitive data** which is **stored** on the disk in **plaintext**

Consequence

Physical access attacks (Reproduce the machine state)

Snapper 2.0

- Configurable number of generations
- Unchanged pages linked, not copied
- ext4 + fsck for crash recovery
- Integrity checks (per-file hashes)
- Configurable redundancy policy

Challenges

- **No prior research** on this for PhantomOS
- **No ready-made solution** exists in Genode

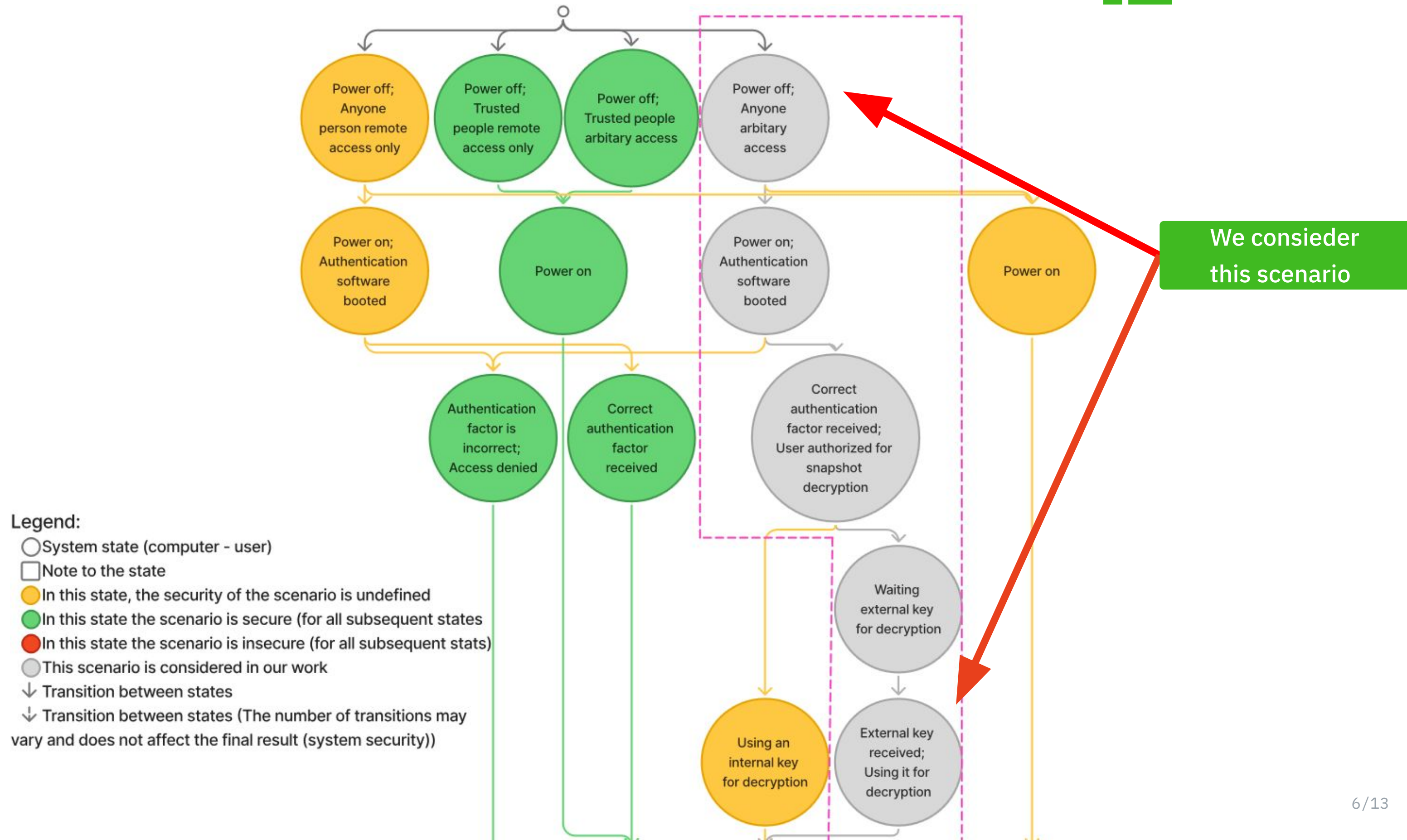
Solution

Implement a mechanism for the secure storage of snapshots.

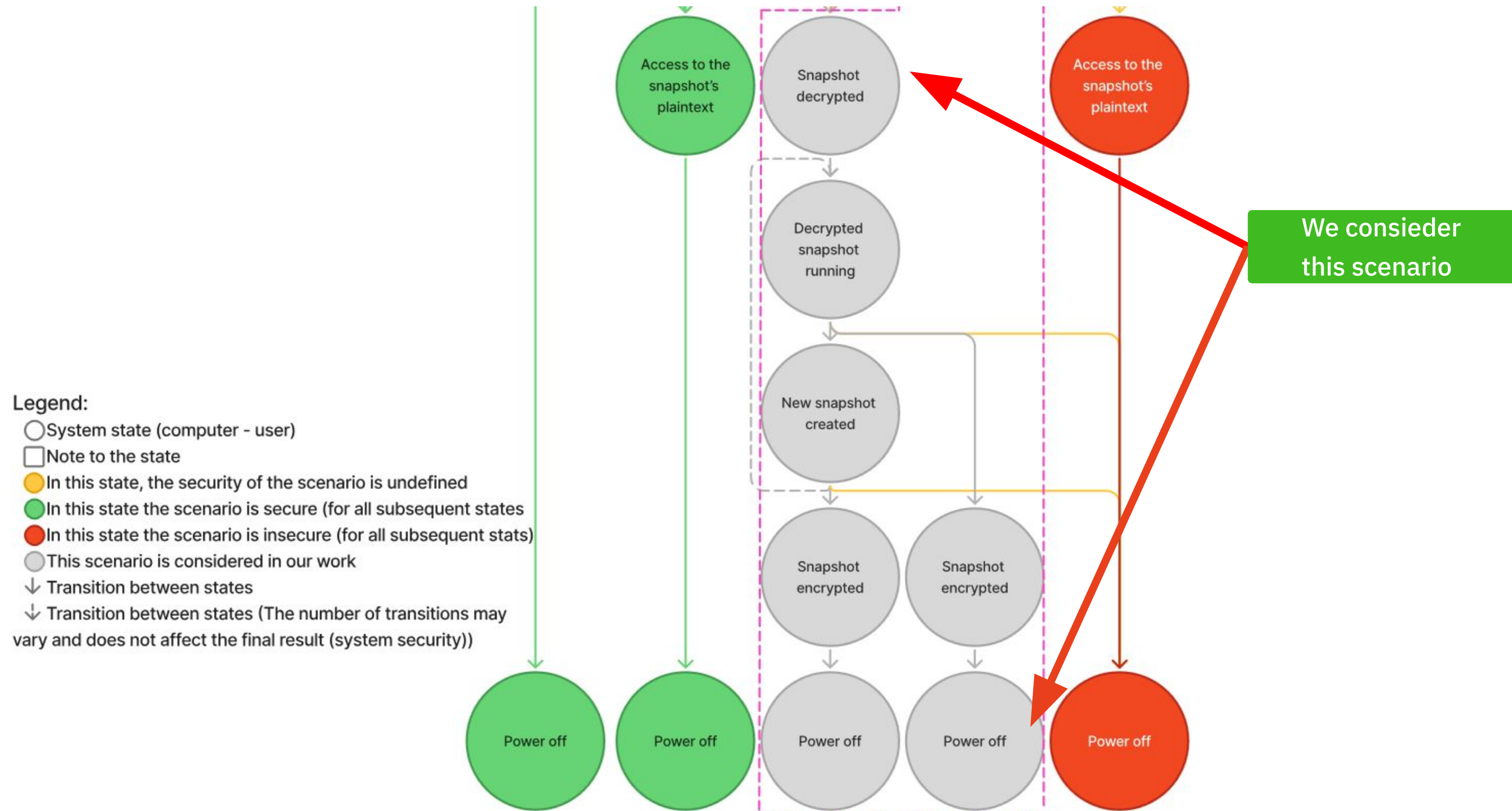
¹<https://github.com/rumenmitov/snapper>

²R. Mitov. “Snapper (v2): A PhantomOS snapshot mechanism”, FOSDEM 2026.

Environment description: Usage scenarios (1/2)



Environment description: Usage scenarios (2/2)



No unauthorized access to data

Target requirements

1. Access control
2. Key stored on an external USB drive
3. Encryption of snapshots (block-level)
4. Only encrypted data written to disk
5. Acceptable performance overhead ($\leq 2\times$)

USB drive = key + auth factor

Requirements 1 and 2 combined: the USB drive both contains the key and serves as the authentication factor.

Implementation

Built on file_vault² (based on Tresor):

- Tresor: AES on 4K blocks, SHA-256 integrity, CoW atomicity
- file_vault: state orchestrator providing a secure FS session

Implemented se_vault¹ as modified file_vault

Changes:

- Headless operation
- Passphrase replaced by **USB key file**
- rump_fs/ext2 replaced by **lwext4/ext4**
- Added **fsck** crash-recovery support
- Direct block session (skip image file)
- All block sizes standardized to 4K
- Graceful shutdown sequence added

Related fixes & Adaptations:

- lwext4³ (ftruncate and sync)
- isomem⁴ (power-off button callback and graceful shutdown)
- snapper⁵ (client counter and graceful shutdown)
- vfs⁶ (client counter, graceful shutdown and clean unmount)

Optimization:

Using a “Block” session directly — initialization time reduced from ≈20 min to ≈1 min

¹https://github.com/genodelabs/genode/tree/sculpt-25.04/repos/gems/src/app/file_vault

²https://github.com/VoronM1522/genode/tree/se/repos/gems/src/app/se_vault

³<https://github.com/VoronM1522/genode-wundertuete>

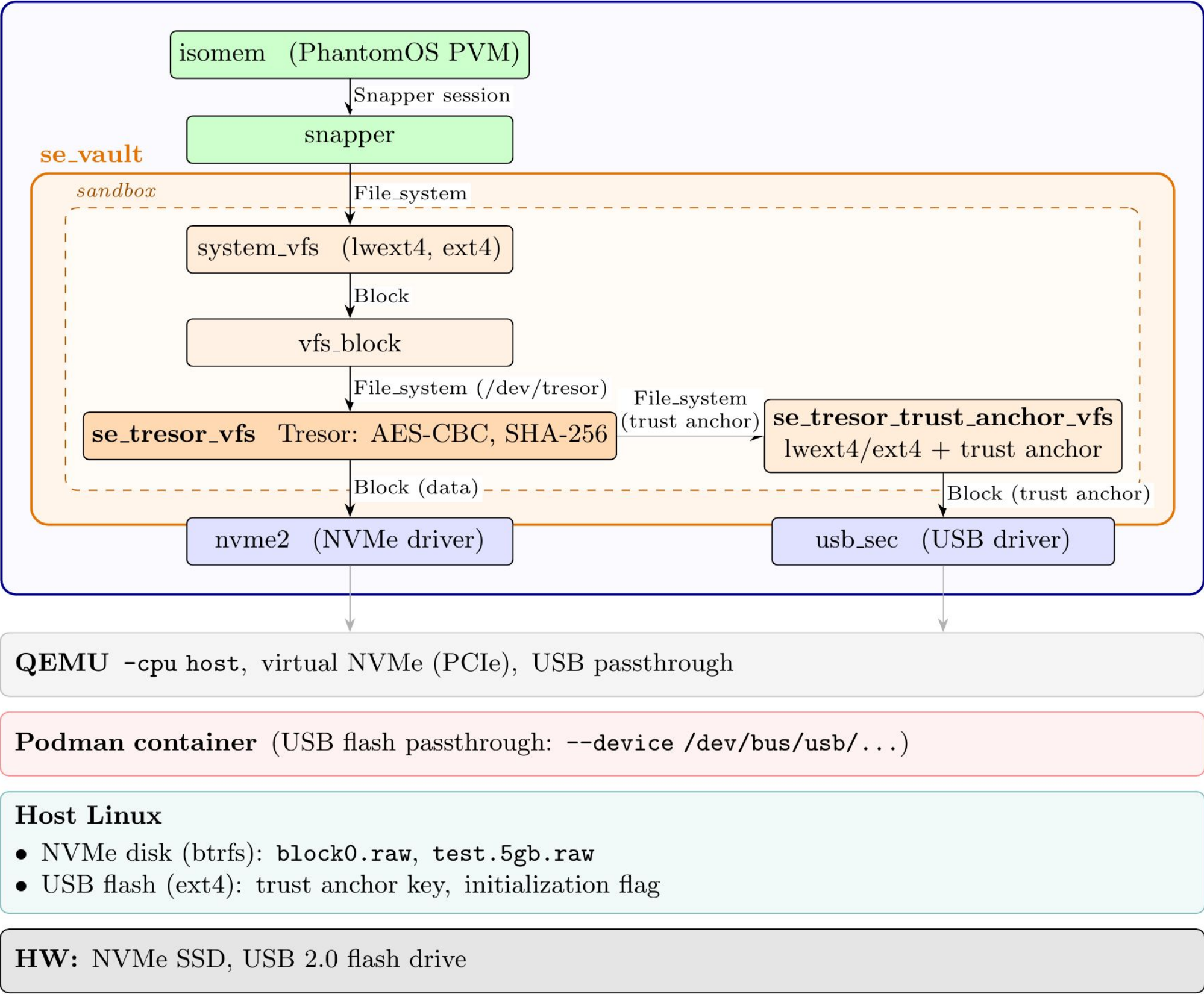
⁴<https://github.com/VoronM1522/phantomuserland-snapper>

⁵<https://github.com/VoronM1522/snapper>

⁶<https://github.com/VoronM1522/genode/tree/se/repos/os>

System Architecture

Genode OS Framework



Implementation results

The system was running on a Linux host inside a Podman container. Virtualized using QEMU with options for AES/SHA hardware acceleration and virtual NVMe usage

All functional checks were passed

- Normal boot / shutdown / restore cycle
- Encrypted image cannot be mounted without key
- Key/hash replacement attack — superblock not detected
- Crash recovery via *fsck*

Performance overhead measurement setup

First snapshot creation and reboot with restore, 5 runs each (*no statistical confirmation*)

Operation	Without enc.	With enc.	Overhead
Snapshot creation	2.10 s	51.00 s	≈ 24×
Snapshot restore	2.50 s	27.30 s	≈ 11×

A performance bug was found during the implementation. Reasons require additional investigation. One of them is a known optimization issue in Tresor

4 of 5 requirements met; Consistent with the considered scenario

- ✓ Mandatory authorization (USB key)
- ✓ Key on external medium
- ✓ AES block-level encryption
- ✓ Only encrypted data on disk
- ✗ Overhead $\leq 2\times$ (Tresor limitation)

The prototype demonstrates feasibility and functional protection, but is not production-ready.

The current overheads may be acceptable in certain scenarios

Improvements

- Security gain: snapshot encryption enabled
- Integrity gain: SHA-256 at block level (Tresor) and hash at FS level (Snapper)

Future work

- Decrease performance overhead (investigate the problem); encrypt the isomem block device
- Merge Snapper and Tresor snapshot concepts;
- Consider other scenarios or additions to the current one, including alternative access options and hardware configurations

Results:

- Computer usage scenarios have been described and classified
- Scenarios requiring encryption and access analysis have been selected
- The applicability of the scenarios was experimentally confirmed based on a mock-up implementation

Objectives of the work mostly achieved. The only issue is a bug in the implementation



INNOPOLIS
UNIVERSITY

Questions

